

SQL VIEW

Ing. Ladislav Körösi, PhD.
ÚRK, FEI STU

ladislav.korosi@stuba.sk

ÚVOD

In SQL, a view is a virtual table based on the result-set of an SQL statement.

A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.

You can add SQL functions, WHERE, and JOIN statements to a view and present the data as if the data were coming from one single table.

CREATE VIEW

Syntax:

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Note: A view always shows up-to-date data! The database engine recreates the data, using the view's SQL statement, every time a user queries a view.

CREATE VIEW

Example:

```
CREATE VIEW [Current Product List] AS  
SELECT ProductID, ProductName  
FROM Products  
WHERE Discontinued = No;  
  
SELECT * FROM [Current Product List];
```

CREATE VIEW

Example:

```
CREATE VIEW [Products Above Average Price] AS  
SELECT ProductName, UnitPrice  
FROM Products  
WHERE  
UnitPrice > (SELECT AVG(UnitPrice) FROM Products);  
  
SELECT * FROM [Products Above Average Price];
```

CREATE VIEW

Example:

```
CREATE VIEW [Category Sales For 1997] AS  
SELECT DISTINCT  CategoryName,  Sum(ProductSales)  
AS CategorySales  
FROM [Product Sales for 1997]  
GROUP BY CategoryName;
```

```
SELECT * FROM [Category Sales For 1997];
```

```
SELECT * FROM [Category Sales For 1997]  
WHERE CategoryName = 'Beverages';
```

UPDATE

Syntax:

```
CREATE OR REPLACE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

UPDATE

Example:

```
CREATE OR REPLACE VIEW [Current Product List] AS  
SELECT ProductID, ProductName, Category  
FROM Products  
WHERE Discontinued = No;
```


DROP

Syntax:

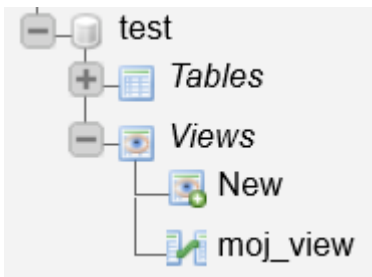
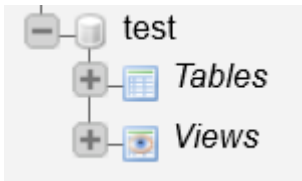
```
DROP VIEW view_name;
```

PHPMYADMIN

```
CREATE VIEW moj_view AS  
SELECT nazov, rok  
FROM filmy  
WHERE imdb>5
```

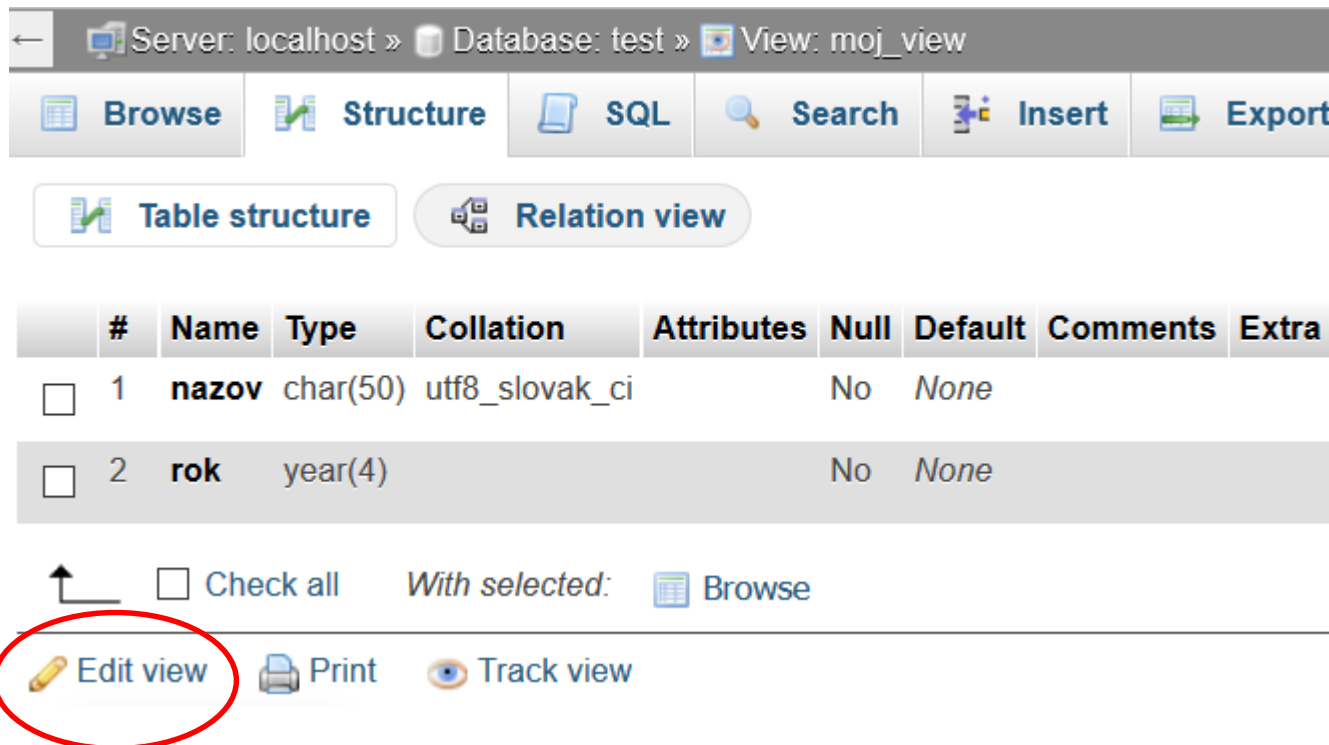
PHPMYADMIN

Zoznam Views je pod Tables:



Klik na moj_view alebo
`SELECT * FROM moj_view`

Editácia View (Edit View alebo SQL príkaz):



The screenshot shows the PHPMyAdmin interface for editing a view. The breadcrumb navigation at the top indicates the path: Server: localhost » Database: test » View: moj_view. Below this is a toolbar with buttons for Browse, Structure, SQL, Search, Insert, and Export. Under the Structure button, there are two sub-buttons: Table structure (selected) and Relation view. A table listing the view's columns is displayed with columns: #, Name, Type, Collation, Attributes, Null, Default, Comments, and Extra. The table contains two rows: row 1 with name 'nazov', type 'char(50)', collation 'utf8_slovak_ci', and row 2 with name 'rok', type 'year(4)'. Below the table, there is a 'Check all' checkbox and a 'With selected:' dropdown menu set to 'Browse'. At the bottom, there are three buttons: 'Edit view' (circled in red), 'Print', and 'Track view'.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐ 1	nazov	char(50)	utf8_slovak_ci		No	None		
☐ 2	rok	year(4)			No	None		

↑ ☐ Check all With selected:

SHOW CREATE VIEW moj_view

Nový View (New cez PhpMyAdmin):

Create view

Details

OR REPLACE

ALGORITHM

Definer

SQL SECURITY

VIEW name

Column names

AS

WITH CHECK OPTION

1

☐

UNDEFINED

Go

Close

View processing algorithms

The algorithm attribute allows you to control which mechanism MySQL uses when creating the view. MySQL provides three algorithms: MERGE, TEMPTABLE, and UNDEFINED.

Using **MERGE** algorithm, MySQL first combines the input query with the SELECT statement, which defines the view, into a single query. And then MySQL executes the combined query to return the result set. The MERGE algorithm is not allowed if the SELECT statement contains aggregate functions such as MIN, MAX, SUM, COUNT, AVG, etc., or DISTINCT, GROUP BY, HAVING, LIMIT, UNION, UNION ALL, subquery. In case the SELECT statement refers to no table, the MERGE algorithm is also not allowed. If the MERGE algorithm is not allowed, MySQL changes the algorithm to UNDEFINED. Note that the combination of input query and the query in the view definition into one query is referred to as view resolution.

Using **TEMPTABLE** algorithm, MySQL first creates a temporary table based on the SELECT statement that defines the view, and then it executes the input query against this temporary table. Because MySQL has to create a temporary table to store the result set and moves the data from the base tables to the temporary table, the TEMPTABLE algorithm is less efficient than the MERGE algorithm. In addition, a view that uses TEMPTABLE algorithm is not updatable.

Using **TEMPTABLE** algorithm, MySQL first creates a temporary table based on the SELECT statement that defines the view, and then it executes the input query against this temporary table. Because MySQL has to create a temporary table to store the result set and moves the data from the base tables to the temporary table, the TEMPTABLE algorithm is less efficient than the MERGE algorithm. In addition, a view that uses TEMPTABLE algorithm is not updatable.

PHPMYADMIN

Full syntax:

```
CREATE
  [OR REPLACE]
  [ALGORITHM = {UNDEFINED | MERGE |
  TEMPTABLE}]
  [DEFINER = { user | CURRENT_USER }]
  [SQL SECURITY { DEFINER | INVOKER }]
  VIEW view_name [(column_list)]
  AS select_statement
  [WITH [CASCADED | LOCAL] CHECK OPTION]
```

DEFINER | INVOKER

- Definer - s právmi toho, kto vytvára
- Invoker – s právmi toho, kto SQL volá

In a WITH CHECK OPTION clause for an updatable view, the LOCAL and CASCADED keywords determine the scope of check testing when the view is defined in terms of another view. The LOCAL keyword restricts the CHECK OPTION only to the view being defined. CASCADED causes the checks for underlying views to be evaluated as well. When neither keyword is given, the default is CASCADED.